

Relatório de Aprendizado - Transfer Learning

Tema: Treinamento de Redes Neurais com Transfer Learning

Ambiente: Google Colab (Python)

Johnnie Richard Pereira

1. Introdução

A ideia deste documento é fazer uma descrição geral sobre meu processo de aprendizado sobre Transfer Learning, uma das técnicas mais importantes do Deep Learning moderno. Essa atividade faz parte do curso sobre Machine Learning da plataforma DIO. O objetivo não é apenas registrar o que o código faz, mas explicar por que cada decisão foi tomada, de modo a refletir, por exemplo, sobre os erros cometidos ao longo de todo percurso de criação do código. Acredito que essa reflexão sobre os erros cometidos ao longo da programação seja uma parte fundamental do caminho de aprendizagem, pois estou no início da minha jornada em Machine Learning e Python. Logo, esse documento reflete meu entendimento atual, que pode conter simplificações ou imprecisões. Por este motivo, deixo claro que correções e sugestões são bem-vindas!

2. O que eu entendia antes de começar

Antes de estudar este tema, meu entendimento sobre redes neurais era bastante superficial, isto para não dizer nulo. Não sabia, por exemplo, que as redes neurais "aprendem com dados". De certo modo, entendia que havia algum tipo de processo de aprendizagem, mas não tinha conhecimento para explicar como esse processo ocorria. Além disso, não entendia o que acontecia matematicamente durante o processo. Talvez, poderia chutar que havia algum tipo de método estatístico envolvido, mas, ao mesmo tempo, não fazia ideia do que era uma "camada convolucional"¹. Por fim, achava que treinar um modelo sempre exigia milhões de dados.

¹ De modo geral, trata-se do bloco fundamental de uma Rede Neural Convolucional (CNN), projetada especificamente para processar dados estruturados em grade, como imagens (2D) ou sinais de áudio (1D)

3. O que aprendi: conceito por conceito

3.1 Redes Neurais Convolucionais (CNNs)

As chamadas Redes Neurais Convolucionais (CNNs) são redes especializadas em processar imagens. Em vez de olhar para todos os pixels de uma vez, elas aplicam filtros pequenos (chamados de kernels) que deslizam pela imagem para detectar padrões. Em uma analogia mais simples, imagine uma lanterna pequena varrendo uma foto no escuro. Supondo que você possa interagir com essa lanterna, ao varrer essa imagem a pequena lanterna te questiona se: "tem uma borda aqui? Tem uma textura? Tem uma curva?". Daí, ao longo desse processo, as primeiras confirmações permitem que seu sistema "lanterna-pessoa" consiga detectar padrões simples (bordas, cores e etc.), enquanto que as camadas do meio detectam formas (olhos, rodas), e as últimas detectam objetos inteiros (rostos, carros e etc.).

```
python

# Cada Conv2D aplica vários filtros para detectar padrões diferentes
# 'filters=32' significa que a camada aprende 32 padrões distintos
# 'kernel_size=(3,3)' é o tamanho da "lanterna"
Conv2D(filters=32, kernel_size=(3, 3), activation='relu')
```

3.2 Transfer Learning

Em linhas gerais, podemos considerar que o Transfer Learning visa reaproveitar uma rede neural já treinada em um grande dataset para resolver um problema diferente, mas relacionado. Desse modo, ao reutilizar os padrões que uma rede aprende para reconhecer objetos em geral (bordas, texturas, formas) economiza-se tempo (e processos computacionais), pois foca-se apenas nas especificidades do novo aprendizado a ser desenvolvido.

É nesse contexto que destaca-se o modelo VGG16. O VGG16 é uma arquitetura clássica de Rede Neural Convolucional (CNN), amplamente utilizada em Deep Learning para tarefas de visão computacional, especialmente classificação de imagens e detecção de objetos. De forma didática, podemos imaginar o seguinte cenário: ao invés de treinar um chef de cozinha do zero (que não sabe absolutamente nada sobre culinária) até chegar a sua máxima excelência em culinária brasileira, a empresa decide contratar um chef experiente de culinária italiana para cozinhar comida brasileira. Ele já sabe segurar a faca, controlar o fogo,

equilibrar sabores. Ele só precisa aprender receitas novas que pertencem ao universo gastronômico brasileiro, isto é, não precisa aprender a cozinhar do zero.

```
python

# include_top=False → não inclui a "cabeça" classificadora original
# weights='imagenet' → carrega os pesos aprendidos no ImageNet
base_model = VGG16(weights='imagenet', include_top=False,
                    input_shape=(224, 224, 3))

# Congela todos os pesos: eles não mudam durante o treino
base_model.trainable = False
```

3.3 A nossa rede base: aprofundando nossos conhecimento no VGG16

A história do VGG16 começa por volta de 2014. Trata-se de uma rede neural criada pela Oxford que, em 2014, foi campeã do ImageNet. Ao todo, estimasse que a VGG16 possui 16 camadas com parâmetros (13 convolucionais + 3 densas) e foi treinada em 1,2 milhão de imagens de 1.000 categorias. A vantagem de aprender usar o Transfer Learning com o VGG16 é que essa tecnologia possui uma arquitetura linear e sem bifurcações, o que facilita muito o entendimento. Além disso, há uma boa documentação base para consulta. Evidentemente, existiam outros caminhos possíveis de serem seguidos (opções mais modernas como ResNet, EfficientNet) que podem ter resultados melhores, mas o VGG16 é ótima para aprender os conceitos necessários no curso. Logo, por simplicidade, optamos por esse caminho.

Para o treinamento requerido pelo projeto, nós fizemos algumas alterações simples para alcançar o resultado esperado. Por exemplo, removemos as 3 últimas camadas (que classificavam 1.000 categorias), congelamos todos os pesos das camadas restantes e adicionamos nossas próprias camadas no final (para 7 classes).

3.4 Data Augmentation: Como "enganar" o modelo de forma saudável

Podemos definir o conceito de Data Augmentation como um processo que visa aumentar artificialmente a quantidade e a diversidade do conjunto de dados de treinamento, sem precisar coletar novos dados. Nesse aspecto, cria-se variações artificiais das imagens de

treino para que o modelo aprenda a ser robusto a diferentes condições (iluminação, ângulo, posição). Como nossa base de dados contava com uma quantidade próxima a 600 imagens por classe, o que é considerado pouco para treinamento de redes neurais, usamos o Data Augmentation para multiplicar virtualmente esse número.

Transformação	O que faz	Por que ajuda
<code>rotation_range=20</code>	Gira a imagem em até 20°	Um pássaro pode aparecer em qualquer ângulo
<code>horizontal_flip=True</code>	Espelha horizontalmente	Um gato olhando para a direita = gato olhando para a esquerda
<code>zoom_range=0.2</code>	Aplica zoom in/out	O animal pode estar longe ou perto
<code>width_shift_range=0.2</code>	Desloca horizontalmente	O animal pode não estar centralizado

Um dado importante de ser registrado é que, de início, apliquei Data Augmentation também no conjunto de validação. Isso estava errado, pois a validação precisa ser com imagens "limpas" para medir a performance real. Ao notar este erro, removi o Data Augmentation do conjunto de validação, deixando-o apenas no treino.

3.5 Overfitting: Quando o modelo "cola na prova"

O conceito de Overfitting pode ser descrito, de maneira lúdica, como sendo um processo de aprendizagem no qual o programa treinado "decora" exemplos específicos em vez de aprender padrões gerais. Isto resulta em resultados muito bons durante o treinamento, mas que falham no reconhecimento de dados novos. Isto aconteceu durante nosso projeto. Para detectar o erro, recorreremos aos gráficos de treinamento. Nele, notamos que a accuracy de treino estava evoluindo bem (subindo progressivamente), mas a curva de validação se mantinha estática, isto é, parou de crescer.

Para combater esse “vício” de aprendizagem, recorreremos a documentação específica e vimos que havia pré estabelecidos alguns passos para contornar essa situação, sendo eles:

- **Dropout(0.5):** Durante o treino, desliga-se 50% dos neurônios aleatoriamente em cada passagem. Isto força a rede a não depender de nenhum neurônio específico;

- **Early Stopping:** Para o treino automaticamente se a validação não melhorar por 5 testes seguidos;
- **Data Augmentation:** As imagens nunca são exatamente iguais entre os testes;

3.6 Fine-Tuning: O refinamento final

Após treinar nossas camadas novas, "descongelamos" as últimas camadas da VGG16 e deixamos que elas se ajustem levemente ao nosso dataset criado. A questão é que as primeiras camadas da VGG16 aprendem padrões muito genéricos (bordas, cores) que servem para qualquer imagem. As últimas aprendem padrões mais específicos (partes de objetos) que podem se beneficiar do ajuste gerado ao longo do treinamento.

```
python

# Fase 1: learning rate normal
optimizer=Adam(learning_rate=1e-4) # 0.0001

# Fase 2 (fine-tuning): 10x menor
optimizer=Adam(learning_rate=1e-5) # 0.00001
```

Uma questão que pode vir a surgir nessa parte é: Por que usar learning rate menor? A resposta é mais simples do que se imagina. Com uma taxa muito alta, poderíamos "apagar" o conhecimento que a VGG16 já tinha. Isto é como estudar demais uma coisa nova e esquecer o que já sabia.

3.7 A Matriz de Confusão: O processo de entender os erros

Em linhas gerais, podemos definir a Matriz de Confusão como sendo uma tabela que mostra não apenas quantos acertos o modelo teve, mas onde ele errou e com o quê ele confundiu. Abaixo há um exemplo disso:

	Previsto				
	cat	dog	bunting	kingfisher	...
Real cat	[45	3	0	0	] ← 45 gatos identificados corretamente
dog	[2	87	0	1	] ← 2 cachorros confundidos com gatos
bunting	[0	0	38	4	] ← 4 pintassilgos confundidos com martim-pescador

Como nosso modelo visava distinguir cachorros, gatos e algumas espécies de pássaros, constatamos por meio da Matriz de Confusão que o modelo treinado confunde mais espécies de pássaros entre si do que confunde pássaros com gatos e/ou cachorros. Este resultado é relativamente coerente, pois gatos e cachorros são visualmente muito diferentes de pássaros, mas distinguir espécies de pássaros é mais difícil.

4. Erros e dificuldades que enfrentei

4.1 Dataset com nomes em italiano

Um dos problemas que detectamos ao utilizar o dataset para reconhecimento de cachorros e gatos foi que o pacote *Animals-10* usa nomes de pastas em italiano. No entanto, ao longo da programação, adotei a escrita dos nomes dos animais em inglês. Desse modo, ao tentar acessar as pastas (cat) e (dog) não encontrava nada.

Como resultado da correção desse erro, foi necessário a implementação de um mini dicionário de mapeamento que traduz os nomes originais para os nomes das classes que usei no projeto. Isto fica melhor explicado na imagem abaixo:

python

```
python

FOLDER_MAP = {
    'gatto': 'cat',    # gatto = gato em italiano
    'cane': 'dog',    # cane = cachorro em italiano
}
```

4.2 Pássaros não estavam no Animals-10

Outro problema que precisou ser superado, ao longo da implementação do dataset, foi que as espécies de pássaros que pensei inicialmente usar para treinar o modelo não estavam disponíveis no pacote *Animals-10*. A princípio, minha ideia era usar no modelo implementado araras, pavões e flamingos, mas o dataset só tinha animais terrestres comuns. Desse modo, se fez necessário combinar dois dataset diferentes, a saber: o já citado *Animals-10* para mamíferos e *CUB-200-2011* (da Caltech) para pássaros.

4.3 Dataset desbalanceado

Por fim, encontramos um problema quanto à quantidade de dados disponíveis em cada um dos Dataset. O *Animals-10* tinha aproximadamente 4.800 imagens de cachorros, enquanto que o *CUB-200* tinha algo em torno de 60 imagens por espécie de pássaro. Com essa diferença enorme, o modelo tenderia a sempre prever "cachorro" para maximizar a acurácia. Logo, foi necessário criar um gatilho para limitar os dados de gatos e cachorros a 600 imagens cada, balanceando com os pássaros.

5. Comparação: código oferecido pela DIO vs. minha versão

Aspecto	Código original	Minha versão
Dataset	CalTech-101 (97 classes)	Animals-10 + CUB-200 (7 classes)
Data Augmentation	Não tinha	Sim (rotação, zoom, flip)
Estratégia	Feature extraction	Feature extraction + Fine-Tuning
Callbacks	Não tinha	EarlyStopping, Checkpoint, ReduceLR
Avaliação	Acurácia simples	Acurácia + Matriz de Confusão + Relatório por classe
Classes	97 categorias genéricas	2 mamíferos + 5 espécies de pássaros

6. Resultados obtidos

Fase	Acurácia treino	Acurácia validação
Feature Extraction (15 épocas)	~85%	~78%
Fine-Tuning (10 épocas)	~90%	~82%

Podemos dizer que nosso projeto teve uma acurácia final nos testes praticados de aproximadamente 80% . Dos testes feitos, as classes com melhor performance foram cachorros e gatos. Talvez, isso se deve ao fato de serem espécies mais simples de distinguir visualmente. Por outro lado, as classes de pássaros utilizados apresentaram maior dificuldade para o aprendizado de máquina. Apesar de selecionarmos espécies bem diferentes entre si, tais como: Pintassilgo colorido (painted bunting), Martim-pescador (Pied Kingfisher), Cotovia (Horned Lark), Pardal (House Sparrow), Maritaca-amarela (Common Yellowthroat), o modelo não teve tanta facilidade para distinguir cada espécie de pássaros entre si.

7. Recursos que me ajudaram

Recurso	O que aprendi com ele
CS231n — Stanford	Fundamentos de CNNs explicados de forma visual
Keras Transfer Learning Guide	Como implementar fine-tuning na prática
3Blue1Brown — Neural Networks	Intuição visual sobre o que acontece dentro de uma rede neural
Notebook do professor	Estrutura base e referência de implementação

8. Considerações finais

O principal aprendizado proporcionado por este projeto não se restringe aos aspectos técnicos da implementação, mas ao entendimento de que o desenvolvimento de modelos de Machine Learning envolve um processo iterativo de investigação, validação e correção. Problemas como inconsistências no dataset em italiano, registros inválidos de espécies e o desbalanceamento entre classes evidenciaram a importância da análise crítica dos dados e da

compreensão metodológica das etapas do pipeline², em vez da simples reprodução de código pronto.

Além disso, a utilização de Transfer Learning demonstrou uma característica central da área de Inteligência Artificial contemporânea: o avanço do campo ocorre de forma cumulativa e colaborativa. Em vez de reconstruir soluções do zero, modelos pré-treinados permitem aproveitar conhecimento previamente consolidado pela comunidade científica e de desenvolvimento. Nesse contexto, a capacidade de adaptar, avaliar e expandir soluções existentes torna-se tão relevante quanto a criação de novos modelos, especialmente em ecossistemas fortemente apoiados por iniciativas Open Source.

9. Referências bibliográficas

SIMONYAN, K.; ZISSERMAN, A. Very Deep Convolutional Networks for Large-Scale Image Recognition. ICLR, 2015.

TORREY, L.; SHAVLIK, J. Transfer Learning. Handbook of Research on Machine Learning Applications, 2010.

Documentação oficial do Keras - Transfer Learning. Disponível em:

https://keras.io/guides/transfer_learning/

CS231n Stanford - Convolutional Neural Networks. Disponível em:

<http://cs231n.stanford.edu/>

² Nesse caso, estamos nos referindo ao processo de automação de tarefas para desenvolvedores onde o código do software passa por etapas de construção, testes e implantação.